

EXPERIMENT 2

Introduction to C Programming – Area of a circle

- Name: Anvika Gadde
- Sap ID- 590033511
- Date:14/08/2026
- Subject: C programming

AIM

Write a C program to calculate the area of a circle and to understand the stages of compilation in C.

Software I Used-

- ★ Operating System – macOS
- ★ Text editor –Vi
- ★ CC compiler
- ★ Terminal

THEORY

How can I run my C program?

There are two main ways to run a c program:

Online/Offline Methods --

A. Offline Methods

1. Dual boot
2. Direct boot
3. Bootable drive

4.Virtual Machine (VM) -

- Type 1(Bare Metal): Runs directly on hardware—e.g. Hyper-V
- Type 2(Hosted): Runs on top of an existing OS--e.g. virtual box (which is used to run kali Linux)

B. Online Methods--

- 1.CoCalc - provides a Linux environment online.
- 2.OnlineGDB
- 3.JDoodle

Different kinds of Text Editors Used are as following--

- Gedit editor
- Vi editor
- Nano editor

SOURCE CODE

```
#include<stdio.h>           //preprocessor directive to include builtin header file
#define PI 3.14             //PPD to define a symbolic constant or macro
int main(void)
{
    int radius;             //variable definition
    float area;
    printf("Enter the radius: "); //printf() is builtin function to display output string
    scanf("%d", &radius);      //scanf() is BIF to get inputs
    area = PI *radius*radius;  //compute
    printf("Area =%f\n" ,area);
    return 0;
}
~
~
~
~
~
~
~
```

EXPLANATION OF THE CODE

S.no	Code	Explanation
1	<code>#include<stdio.h></code>	Preprocessor directive to include built in header file.
2	<code>#define PI 3.14</code>	Preprocessor directive to define a symbolic constant or macro.
3	<code>int main(void)</code>	The main function where execution begins . Void means no arguments.
4	<code>int radius;</code>	Declares an integer variable to store the radius.
5	<code>float area;</code>	Declares a float variable to store the decimal area value.
6	<code>printf("Enter the Radius: ");</code>	Built in function that displays a message to the user.
7	<code>scanf("%d", &radius);</code>	Built in function that reads an integer input from the user.
8	<code>area= PI *radius*radius;</code>	Computes the area using the formula $A= \pi r^2$.
9	<code>printf ("Area =%f\n",area);</code>	Displays the calculated area as a floating point number.
10	<code>return 0;</code>	Indicates successful program termination.

COMPILATION AND EXECUTION

- After writing the code, save and exit vi editor by pressing **Esc**, then type **:wq** and press **Enter**. This takes you back to the terminal .

Then, use the following commands to compile and execute the program:

1. To open/edit the file:

```
vi area.c
```

2. To compile the code:

```
cc area.c -o area
```

This command compiles area.c and creates an executable file named area. If there are any errors or mistakes in the code, the compiler will display them here.

3. To run the program:

```
./area
```

This executes the compiled program. The program will ask the user to enter the radius and then display the calculated area.

OUTPUT

Enter the radius: 5

- Area = 78.500000

```
puppie@Anvikas-MacBook-Air ~ % vi test.c.  
puppie@Anvikas-MacBook-Air ~ % cd ~/Desktop  
cd: no such file or directory: ~/Desktop  
puppie@Anvikas-MacBook-Air ~ % mkdir thur  
mkdir: thur: File exists  
puppie@Anvikas-MacBook-Air ~ % cd thur  
puppie@Anvikas-MacBook-Air thur % vi area.c  
puppie@Anvikas-MacBook-Air thur % cc area.c -o area  
puppie@Anvikas-MacBook-Air thur % ./area  
Enter the radius: 5  
Area =78.500000
```

STAGES OF COMPILATION

When we compile a C program using the command:

```
cc -save-temps area.c
```

The source code passes through 4 stages and generates intermediate files at each stage:

```
area.c → area.i → area.s → area.o → a.out
```

source code	pp source code	assembly code	object file	linked executable file
----------------	-------------------	------------------	-------------	---------------------------

1. Preprocessing Stage (PP) -> area.i

- Header files are expanded (e.g. content files of stdio.h are pasted in)
- All comments are removed
- Macro substitution happens (PI is replaced with 3.14)

2. Compilation stage -> area.s

- The preprocessed code is converted into **Assembly language** (a low level, human readable representation of machine instructions).

3. Assembling stage -> area.o

- Assembly code is converted into machine code(binary), called the object file.

4. Linking stage -> a.out

- The object file is linked with the standard c libraries (which contain the actual code for functions like printf and scanf) to produce the final executable file, a.out .

```

puppie@Anvikas-MacBook-Air ~ % vi test.c.
puppie@Anvikas-MacBook-Air ~ % cd ~/Desktop
cd: no such file or directory: ~/Desktop
puppie@Anvikas-MacBook-Air ~ % mkdir thur
mkdir: thur: File exists
puppie@Anvikas-MacBook-Air ~ % cd thur
puppie@Anvikas-MacBook-Air thur % vi area.c
puppie@Anvikas-MacBook-Air thur % cc area.c -o area
puppie@Anvikas-MacBook-Air thur % ./area
Enter the radius: 5
Area =78.500000
puppie@Anvikas-MacBook-Air thur % cc -save-temps area.c
puppie@Anvikas-MacBook-Air thur % ls
a.out area area.bc area.c area.i area.o area.s
puppie@Anvikas-MacBook-Air thur % cat area.i | tail -20
    const char * restrict, va_list);

extern int __sprintf_chk (char * restrict , int, size_t,
    const char * restrict, ...);
extern int __vsprintf_chk (char * restrict , int, size_t,
    const char * restrict, va_list);
# 508 "/Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/usr/include/_stdio.h" 2 3
4
# 62 "/Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/usr/include/stdio.h" 2 3 4
# 2 "area.c" 2

int main(void)
{
    int radius;
    float area;
    printf("Enter the radius: ");
    scanf("%d", &radius);
    area = 3.14 *radius*radius;
    printf("Area =%f\n" ,area);
    return 0;
}
puppie@Anvikas-MacBook-Air thur % █

```

RESULT

The C program to calculate the area of a circle was successfully written using the vi editor, compiled using the cc compiler, and executed in the terminal. The four stages of compilation (preprocessing, compilation, assembling, and linking) were also observed using the `-save-temps` flag .